

Reviewer Pack — Minimal Local Coherence of Vertex Operator Algebras and Circ...

Entry d4127da8cdbb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 20:57:12 UTC

Minimal Local Coherence of Vertex Operator Algebras and Circuit Depth Correlation

Entry ID: d4127da8cdbb **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 20:57:12 UTC

1. Conjecture

Field A (mathematical branch): Vertex Operator Algebra Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every instance of a Boolean circuit φ with n variables, the minimal local coherence ($\text{mlc}(\varphi)$) of its associated vertex operator algebra is linearly correlated with its depth $d(\varphi)$, such that $\text{mlc}(\varphi) = \Theta(d(\varphi))$.

Rationale (proposer's reasoning):

Vertex operator algebras provide a categorical framework for studying quantum entanglement and have been used to construct topological phases. The minimal local coherence, which measures the degree of entanglement in the algebra, might correlate with circuit depth as a proxy for complexity.

Taxonomy category: VertexOperatorAlgebraToBooleanCircuitComplexityCorrelation (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c152bfd30f6c2369

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between minimal local coherence ($\text{mlc}(\varphi)$) and depth ($d(\varphi)$) for at least 24 out of 30 seeds is within 0.8 to 1.2, and no seed produces a correlation coefficient outside this range. The conjecture is falsified if any seed produces a correlation coefficient significantly different from 0.8 to 1.2 or if the mean absolute difference between $\text{mlc}(\varphi)$ and $d(\varphi)$ exceeds 3 for at least half of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "vertex operator algebra" AND "Boolean circuit complexity" AND minimal local coherence" - "mlc(ϕ)" AND "depth d(ϕ)" AND vertex operator algebra" - "circuit depth correlation" IN title AND minimal local coherence

Top relevant hits considered: - [<http://arxiv.org/abs/2507.16123v3>] Einstein's Electron and Local Unitary Branching: Boundaries of Islands of Coherence and Quantum Nonlocality - [<http://arxiv.org/abs/1702.06061v4>] Generalized Coherence Concurrence and Path distinguishability - [<http://arxiv.org/abs/1702.03219v4>] Coherence number as a discrete quantum resource

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_circuit(n):
        circuit = []
        for _ in range(random.randint(1, 5)):
            gate = random.choice(['AND', 'OR', 'NOT'])
            if gate == 'NOT':
                qubit = random.randint(0, n-1)
                circuit.append((gate, qubit))
            else:
                qubits = sorted(random.sample(range(n), 2))
                circuit.append((gate, qubits[0], qubits[1]))
        return circuit

    def depth_of_circuit(circuit):
        if not circuit:
            return 0
        max_depth = 0
        for gate in circuit:
            if gate[0] == 'NOT':
                max_depth = max(max_depth, depth_of_circuit([gate[1]]))
```

```

        else:
            max_depth = max(max_depth, depth_of_circuit([gate[2], gate[3]]))
    return 1 + max_depth

def minimal_local_coherence(circuit):
    n = len(circuit)
    mlc = [0] * n
    for i in range(n):
        if circuit[i][0] == 'NOT':
            mlc[circuit[i][1]] += 1
        else:
            qubits = sorted([circuit[i][2], circuit[i][3]])
            mlc[qubits[0]] += 1
            mlc[qubits[1]] += 1
    return sum(mlc) / n

instances_tested = 0
total_mlc = 0
total_depth = 0
max_n = 0

for _ in range(30):
    n = random.randint(5, 40)
    circuit = generate_random_circuit(n)
    mlc = minimal_local_coherence(circuit)
    depth = depth_of_circuit(circuit)

    instances_tested += 1
    total_mlc += mlc
    total_depth += depth
    max_n = max(max_n, n)

mean_mlc = total_mlc / instances_tested
mean_depth = total_depth / instances_tested

correlation_coefficient = (instances_tested * sum(mlc * depth for mlc, depth in zip(total_mlc,
total_mlc * total_depth) / math.sqrt(
    instances_tested * sum(mlc**2 for mlc in total_mlc) - total_mlc**2 *
    instances_tested * sum(depth**2 for depth in total_depth) - total_depth**2))

conjecture_holds = 0.8 <= correlation_coefficient <= 1.2
counterexample = "" if conjecture_holds else f"Correlation coefficient: {correlation_coefficient}"

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": max_n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
    results.append(result)

mean_mlc = sum(r["metric_value"] for r in results) / len(results)
std_mlc = math.sqrt(sum((r["metric_value"] - mean_mlc)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if 0.8 <= r["metric_value"] <= 1.2) / len(results)

if all(0.8 <= r["metric_value"] <= 1.2 for r in results):
    print(f"RESULT: SUPPORTED mean={mean_mlc} std={std_mlc} support_fraction={support_fraction}")
elif any(not (0.8 <= r["metric_value"] <= 1.2) for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not (0.8 <= result["metric_value"] <= 1.2))
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_outside_range\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_add0bd95.py", line 98, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_add0bd95.py", line 64, in run_trial
    mlc = minimal_local_coherence(circuit)
          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_add0bd95.py", line 51, in minimal_local_coherence
    qubits = sorted([circuit[i][2], circuit[i][3]])
                  ~~~~~^
IndexError: tuple index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification based on the pre-registered criteria. | next: Investigate and fix the crash in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24547
2	preregistration	ollama_remote	glm4:latest	0	0	13703
3	novelty	ollama_remote	glm4:latest	0	0	8809
4	novelty	ollama_remote	glm4:latest	0	0	17782
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14667
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24290
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18418
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15883
9	judge	ollama_remote	glm4:latest	0	0	16860

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154960 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d4127da8cddb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d4127da8cddb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d4127da8cddb.tar.gz` (if generated)